

Krishi ❖❖❖ detailed interview answers

Krishi — detailed interview answers

Use these as speaking notes, not a script. Be precise about the distinction between what the prototype **does now** and what you would build next.

1. Product, users, and scope

What problem does Krishi solve, and who is it for?

Krishi is a multilingual agricultural guidance portal for farmers in Dakshina Kannada. Its primary user is a local farmer or farm household member who needs practical, understandable guidance on crop choice, drainage, soil health, pests, markets, and support schemes. Rather than making the user search a large, generic agricultural website, the product brings locally relevant information and a question interface into one place.

The core problem is not simply a lack of agricultural information. It is the difficulty of finding advice that fits a high-rainfall coastal district, a farmer's preferred language, and the immediate decision they are making. The application therefore combines a readable guide with contextual retrieval before generating an answer.

Why focus on Dakshina Kannada rather than all of India?

Agronomy is highly location-dependent. Rainfall pattern, lateritic soil, slope, humidity, local crops, post-harvest practices, markets, and institutions all change what good advice looks like. Dakshina Kannada has heavy monsoon rainfall, so drainage is frequently a more urgent concern than irrigation; areca, coconut, pepper, cocoa, paddy, and hill-taluk rubber are locally meaningful examples. An India-wide guide would either be vague or risk recommending practices that do not fit this district.

The narrow scope is also an engineering decision. A smaller, reviewed corpus makes grounding and content maintenance feasible. Once the content, review process, and retrieval quality are strong for one district, the architecture can scale through district-tagged guide collections rather than pretending one guide applies everywhere.

What are the main user journeys?

1. A farmer chooses English, Kannada, or Tulu and reads the locally curated Markdown guide.
2. They ask a free-text question or tap a suggested question.
3. The browser retrieves up to three relevant sections from the selected language guide.
4. The client sends the question, language, and retrieved excerpts to `/api/ask`.
5. The server calls the configured LLM without exposing the API key, and the UI displays the answer with the retrieved section headings. Users can report an answer as outdated, unsafe, or unclear; the report is linked to its guide release and retrieved sections.

Important qualification: the current Guide/Latest toggle only changes visual state. Both modes presently use the API route. I would say this directly rather than describe an unimplemented feature as working.

Which topics does the guide cover, and how was it structured?

The guide follows a farmer decision flow: district climate and soil context; paddy and its seasonal calendar; plantation and intercrop options such as areca, coconut, pepper, cocoa, rubber, vanilla, banana, fruit, and vegetables; drainage and soil health; pest management and post-harvest handling; institutional support, markets, diversification, and climate adaptation. English additionally includes schemes, market planning, technology, and sustainability.

It uses level-two Markdown headings because they make the guide easy to read and also provide a natural retrieval unit. Each chunk remains coherent enough for an LLM to use: for example, the pepper section contains both suitability and drainage/disease advice. The limitation is that Kannada and Tulu currently stop at section 25 while English has sections 26–29, so guide parity must be fixed.

Why English, Kannada, and Tulu? What does Tulu add?

English supports users comfortable with formal or technical information; Kannada is the state language and broadens accessibility; Tulu acknowledges a major local spoken language and can reduce the friction of asking in the language people use at home and on farms. Adding Tulu is not merely a translation feature: it makes the product feel locally owned and can include users who may not be comfortable formulating a detailed agricultural question in English.

Translations should be maintained from one source-of-truth content matrix. Each section needs a stable ID, approved terminology, reviewer sign-off, and a parity test that ensures all language versions contain the same required sections. Back-translation and bilingual subject-matter review are more useful than relying only on machine translation.

What does “locally grounded” mean, and how is this different from a generic chatbot?

It means the answer is informed by a curated Dakshina Kannada guide before the LLM is called. The client retrieves semantically similar local sections, supplies them as context, and labels their headings in the response UI. A generic chatbot begins from broad model knowledge and may not know which advice is suitable for coastal Karnataka.

Grounding is helpful, but the current product does not enforce it strictly. The server prompt permits general-knowledge answers when context is missing. It also does not retrieve live web sources, so “latest information” and “citations” are not yet accurate product claims.

Requirements, reliability, and success measures

Functional requirements are multilingual guide rendering, language switching, semantic/keyword retrieval, persistent local vectors, suggested questions, chat requests, a server-side key boundary, versioned guide releases, and answer feedback. Non-functional requirements are responsive performance, graceful fallback when browser ML or IndexedDB fails, privacy-conscious data handling, accessible controls, low-bandwidth resilience, consistent multilingual content, and safe content updates without an availability gap.

The static guide has independent value: it remains readable even when the LLM, embedding model, API key, or network is unavailable. Suggested questions reduce the blank-page problem, demonstrate useful topics, and help users ask questions that the current corpus can answer well.

The app now includes feedback categories for **outdated**, **unsafe**, and **unclear**, plus an optional reviewer comment. Each report records the answer ID, guide version, language, and retrieved section IDs, so a reviewer can trace it to the exact release. I would also measure no-match and escalation rates, repeat-question rate, time to answer, expert-labelled retrieval relevance, and follow-up completion. These metrics should be anonymized and never used to infer sensitive farm economics without consent.

Assumptions, limitations, and next two weeks

The prototype assumes a modern smartphone or desktop browser with JavaScript, IndexedDB, Web Crypto, and enough memory/network to download an embedding model at least once. It assumes that users can read one of the three languages or can seek help from a household member. Those are assumptions to validate, not guarantees.

Current limitations include client-side model download and first-use latency, uneven language guides, no visible request error/retry state, no tests, no rate limiting or upstream timeout, no live retrieval, and no expert review workflow encoded in the product. Feedback is persisted to a local NDJSON file for a single development server; production needs a shared database or review-ticket service.

With two more weeks I would: (1) make Guide mode local-only and make Latest mode genuinely source-backed; (2) return stable source IDs, excerpts, dates, and confidence; (3) add visible errors/retry and a local fallback; (4) complete language parity; (5) add rate limits, upstream timeouts, tests, and a durable feedback store; and (6) have an agronomist review high-risk content and escalation copy.

2. Front-end architecture and accessibility

Why React and Vite? What is App.jsx responsible for?

React fits the interactive state model: selected language, chat visibility, typed question, messages, selected mode, request activity, and index readiness. Vite gives fast local development, raw-content imports, and an efficient production build with a simple configuration. `App.jsx` currently orchestrates guide selection, section parsing, vector indexing/retrieval, API requests, and all UI rendering.

One component was acceptable for an early prototype because the behavior is tightly coupled and easy to demonstrate. It is no longer ideal: retrieval logic and a large UI tree make it hard to test or change independently. I would extract `LanguageSwitcher`, `Guide`, `ChatPanel`, `MessageList`, `ChatComposer`, `ModeSwitcher`, and a `lib/retrieval.js` module first. `ChatPanel` and retrieval are the highest-value extractions because they contain the most stateful behavior and future features.

Which hooks are used and why?

`useState` owns mutable UI state: `lang`, `chatOpen`, `question`, `messages`, `mode`, `asking`, and `indexStatus`. These values directly affect rendering and are local to the app. `useMemo` derives parsed guide sections from the selected guide; because a guide string changes only when `lang` changes, `[lang]` is sufficient and avoids reparsing on every keystroke or message update.

`useEffect` prepares or validates the selected guide's vector index after `lang` or `sections` changes. Its cancellation flag prevents a completed asynchronous operation from setting status after an effect is superseded. It does not cancel the underlying model/index operation; a more mature implementation would support cancellation where possible.

Changing language recomputes sections, starts indexing for that guide, changes labels/content, and clears messages. Clearing avoids showing answers and source headings from a different language, but it may feel abrupt. Better alternatives are language-specific histories, a confirmation, or retaining the question with a “translate/re-answer in this language” option.

Strict Mode, Markdown, raw imports, and bundle choices

`main.jsx` uses `React.StrictMode`, which intentionally re-runs certain development lifecycles to expose impure effects, deprecated patterns, and unsafe side effects. It does not change production behavior, but it can make indexing calls appear twice in development—another reason operations should be idempotent.

The app renders guide and answer Markdown with `react-markdown` and `remark-gfm`; GFM adds familiar tables, task lists, strikethrough, and similar syntax. This is safer than injecting raw HTML because React Markdown parses Markdown into React elements rather than directly using arbitrary HTML. It should still avoid enabling raw HTML without robust sanitization, and external links need safe policies.

Vite's `?raw` imports compile each Markdown file into a string at build time. Bundling makes content available immediately and offline after the app loads, avoids runtime content fetch errors, and lets Vite fingerprint the build. The tradeoff is a larger initial bundle and loading all language content even when only one is used. `Dynamic import()` can lazy-load guide files; the embedding model can likewise load only after the first non-greeting question, preferably with a clear progress indicator and cache-aware prefetch.

Layout and accessibility

The responsive CSS changes layout at 850px. The chat is sticky beside the guide on wider displays so both remain visible while reading; on smaller screens it becomes fixed so it acts like a compact, reachable assistant rather than consuming a narrow column. Lucide provides consistent SVG icons, but icons need labels because their visual meaning is not universally available. The close/open buttons have `aria-label`, the chat `<aside>` has a label, and the mode buttons are grouped.

Improvements include visible focus states, semantic button state (`aria-pressed` for the mode), a labelled input, `aria-live="polite"` for answer/loading/error status, keyboard-friendly suggested buttons, and focus management. On opening, save the triggering button and focus the composer or chat heading; on closing, return focus to that opener. Do not move focus on every streamed token; announce a concise “Answer ready” status instead.

3. Retrieval and RAG

Explain the end-to-end question path. Why is it RAG-style?

For a non-greeting, `ask()` obtains relevant sections by calling `findSemanticAnswer(question, lang, sections)`. That function embeds the question, compares it with section vectors from `IndexedDB`, keeps up to three sufficiently relevant matches, and uses keyword ranking if semantic retrieval is weak or fails. The client sends only the question, selected language, and matching `{heading, content}` objects to `/api/ask`. The Node server constructs a prompt from that context and requests an LLM completion. This is RAG-style because retrieval augments generation with external, project-owned information at answer time.

Retrieving sections instead of sending the entire guide reduces prompt size, cost, latency, and distraction. It also makes it more likely the model attends to the relevant evidence. The tradeoff is retrieval error: if the right section is missed, the model has incomplete context.

How are chunks built, and why include headings?

`buildSections` splits Markdown before each `##` heading using `/(?=>##\s)/m`. It then retains only chunks whose first level-two heading can be extracted. Any introduction before the first `##` is discarded from retrieval; that is acceptable only if it contains no critical guidance. A production parser could preserve a preface chunk or attach it as guide metadata.

The embedding text is `heading + newline + content`. Headings contain compact domain terms—such as “Water and Drainage Management”—that make a section easier to retrieve for short questions. They also provide a human-readable source label.

Model, embeddings, and cosine similarity

`pipeline('feature-extraction', 'Xenova/all-MiniLM-L6-v2')` loads a `Transformers.js` feature-extraction pipeline in the browser. It turns text into a vector representation that supports semantic similarity: terms with related meaning

can be close even if their literal wording differs. Mean pooling combines token-level representations into one vector; `normalize: true` scales it to length one.

Similarity is implemented as a dot product. For unit-normalized vectors, the dot product equals cosine similarity, so no separate magnitude division is necessary. The embedding promise is module-level (`embedderPromise`), so the model is loaded once per page session. Vectors are persisted separately in IndexedDB for future sessions.

Browser-side embeddings keep the guide and question on the device until the later API call, reduce backend vector infrastructure, and make a local fallback possible. Costs are first-use download/compute, varied phone capability, browser compatibility, and weaker control over the retrieval trust boundary. The chosen MiniLM model may not represent Kannada and especially Tulu as well as English; test it with bilingual relevance sets. A multilingual embedding model such as `paraphrase-multilingual-MiniLM-L12-v2` or a newer multilingual retrieval model should be benchmarked before adoption rather than assumed to be better.

Threshold, top-k, quality, and fallback

The `0.32` threshold is a heuristic: when the best semantic match falls below it, the app uses keyword search. It must be tuned with a labelled set of real farmer questions: compare retrieved results to expert relevance labels, then select a value that balances irrelevant context against missed relevant context. Thresholds may need to differ by language and model.

Top-three is a practical context budget. Too few chunks can omit a necessary caveat; too many add irrelevant text, cost, and opportunities for the model to blend contradictory advice. Evaluate retrieval with Recall@3, MRR/nDCG, no-match precision, language-specific results, and error categories. Poor retrieval is likely for vague questions, names/spellings not in the guide, code-switched text, multiple questions, and underrepresented Tulu phrasing.

The keyword fallback tokenizes Unicode letters/numbers using `\p{L}` and `\p{N}`, so it does not assume ASCII. It removes a small English stop-word set, adds a few English related terms, scores heading hits as four and body hits as one, then takes up to three positive sections. Heading weighting emphasizes a deliberately named topic. Its limitations are clear: its stop words and synonym map are English-only, it has no morphology/spelling normalization, and it cannot understand Tulu or Kannada semantic variants well.

IndexedDB, hashes, versioning, and zero-downtime releases

IndexedDB is chosen because vectors are sizeable structured data; `localStorage` is synchronous, string-only, small, and can harm UI responsiveness. Every section has a SHA-256 hash of heading plus content. On load, stored entries are current only if count, heading order, section ID, and each hash match the current sections. This invalidates changed text safely.

The current implementation avoids the old delete-and-rewrite race by never overwriting a release in place. Vector IDs include `version:language:section`, and a release has its own version namespace. The app reads a configured manifest, downloads every language file for the proposed release, builds its indexes, and only then swaps the rendered release. Thus a user sees either the complete old release or the complete new one—not a mixture. The manifest is checked on load and every five minutes.

Publish immutable guide URLs such as `/guides/42/en.md` first, then atomically point the small active manifest to version 42. Keep the manifest uncached or on a short TTL and give immutable guide URLs a long cache lifetime. A rollback is a manifest change back to the prior healthy version; old guide/vector data remains usable. The Service Worker and Cache Storage preserve guide/version assets for offline reuse. This release process is documented in `GUIDE_RELEASES.md`.

An index schema version should still be added when changing vector format, embedding model, chunking, or hash strategy. A scheduled cleanup policy can remove no-longer-referenced release namespaces only after the rollback window expires.

If IndexedDB, Web Crypto, model load, or embedding fails, the catch path uses keyword retrieval. The UI now says “Keyword matching is in use” rather than claiming semantic grounding. Meaningful progress could still report model download percentage, guide vector count, and ready state. First-question latency can be reduced by idle-time preloading, lazy indexing after guide render, a cached model/service worker, and by not making greetings load the model.

For many districts or thousands of documents, client-side embedding becomes expensive. Add metadata (district, crop, source author, review date, language, confidence, document version) and move embedding/search to a backend vector service when corpus size, security, observability, or update frequency requires it. A Service Worker can cache guides, app assets, and a previously downloaded model for better offline use; it should still expose freshness/version status.

4. API, LLM, privacy, and security

Why have a separate Node server? What does the endpoint do?

The browser must never hold `LLM_API_KEY`: any user could inspect it and spend or abuse it. `server.cjs` provides a

small trust boundary and reads `LLM_API_URL`, `LLM_API_KEY`, and optional `LLM_MODEL` from environment variables via `dotenv`. `POST /api/ask` accepts a question, supported language, guide version, and up to three retrieved context sections; it returns an answer and the supplied guide version. `POST /api/feedback` accepts a bounded, structured answer-quality report. During development, Vite proxies `/api` to port 8787, letting the frontend call a same-origin-looking path while the API runs separately.

If URL/key configuration is missing, `generateAnswer` now fails before attempting an upstream request, though the client still needs a clearer visible configuration/error state. A production server should also fail fast at startup with an actionable configuration error. `gpt-4o-mini` is merely the default configured model, not a hard dependency. Temperature 0.1 aims for consistent, low-creativity agricultural responses.

How does prompting and context work? What is the trust issue?

The system prompt asks the model to answer in `language`, use relevant local context, use general knowledge when no context is relevant, and not pretend general information came from the guide. The user message contains the raw question and a concatenation of client-provided headings/content.

This is useful but not a strong trust boundary. A modified browser client can send arbitrary `localContext`, and guide content itself may contain instructions. For a stronger design, send only a guide ID and question; perform retrieval server-side from an approved, versioned corpus; treat retrieved text as data, delimit it, and instruct the model to ignore instructions contained within sources. Strict grounded mode should say “I do not have enough approved source information” rather than use general knowledge.

Citations, validation, feedback, errors, and modes

The headings displayed by the UI are local retrieval labels, not model-verified citations. There are no external current sources. A robust API would return a typed response such as `{answer, sources:[{id, title, excerpt, url, publishedAt, guideVersion}], confidence, mode}` and make citations link to the exact guide section. The current server does validate that the provider returned a non-empty `choices[0].message.content` before returning it; malformed provider responses become a controlled server error.

The feedback flow is intentionally traceable: the browser submits answer ID, guide version, language, retrieved section IDs, one of the allowed categories, and an optional comment. The development server validates and appends the report to `data/feedback.ndjson`, which is ignored by Git. That is sufficient for a single instance/demo, but it is not a durable multi-instance production store.

The API explicitly returns 400 only for an empty question; all parse, provider, and shape failures become 500. Add 400 for invalid JSON/fields, 413 for oversized body, 429 for rate limits, 502 for a malformed/unavailable provider, and 504 for timeout. The client currently logs an error but displays nothing; it should preserve the question, show an error and retry button, and offer the local retrieved excerpts when possible.

The Guide/Latest mode must be made real. Guide mode should return only cited local results—possibly an extractive answer—without an API call. Latest mode must use approved live sources such as official scheme, weather, or market feeds and disclose their timestamps. Until then, rename Latest to something truthful or remove it.

Production hardening and operations

The server now bounds JSON request bodies to 64 KB; checks question presence/length, supported language (`en`, `kn`, `tcy`), local-context array count and field sizes; and returns 400 for invalid JSON/fields or 413 for oversized bodies. The client disables both the composer and suggested questions while a request is active. Remaining hardening includes per-IP/user rate limits, request IDs, structured logs with redaction, and caching keyed by normalized question + language + guide/source version—never cache private conversational details indiscriminately.

Use upstream timeouts and bounded retries only for safe transient failures; support cancellation when the client disconnects. Streaming can be implemented with Server-Sent Events or a streamed fetch response, appending token deltas in the UI while preserving accessibility. Conversation history improves follow-up answers but increases cost, prompt-injection surface, and privacy exposure, so retain only user-consented, bounded history.

In production, serve the Vite build from a static host/CDN and run the API behind HTTPS reverse proxying. Configure an allow-list CORS origin, CSP, HSTS, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, and clickjacking protection. Monitor latency, 4xx/5xx rate, provider status, tokens/cost, retrieval confidence, no-match rate, and feedback. Deploy content/model changes with versioned manifests and a rollback target.

Tell users exactly what is sent to the provider: their question, selected language, and retrieved guide excerpts; do not send farm identity or location unless required and consented. Provide a concise privacy notice, obtain consent where appropriate, minimize/strip PII such as phone numbers and addresses, retain logs briefly, and offer an anonymous guide-only mode.

5. Current gaps: short, candid answers

- **Does the mode switch alter behavior?** No. `ask()` always calls `askLatest()`, which calls the LLM API.

Implement a local-only Guide branch and a verified-source Latest branch.

- **Are citations current/verifiable?** No. They are section headings from local matches. The app does not perform web/search retrieval.
- **What happens after request failure?** The console logs an error and clears loading; the user still sees no error, retry, or fallback. Add all three.
- **Are simultaneous requests prevented?** Yes for normal UI use: composer and suggested-question controls are disabled while a request is active. Server-side idempotency/request sequencing would add defence in depth.
- **Is server validation sufficient?** It now enforces JSON/body limits, question length, supported language, context shape/count/field sizes, feedback fields, and provider response shape. It still needs rate limiting, authentication where appropriate, upstream timeout/retry, and a stronger server-side retrieval trust boundary.
- **Why is `updatedAt` rendered?** It is unfinished: no message assigns it, so the branch never renders.
- **Why is `labels.empty` unused?** It appears intended for no-match feedback, but the app calls the server even with no local match.
- **Is `App.css` used?** No. `main.jsx` imports `index.css`; remove or integrate stale CSS after confirming it is unused.
- **Is the README production-ready?** No. It is largely Vite boilerplate and needs setup, environment variables, API process, architecture, safety, tests, and deployment documentation.
- **Are there tests?** No test script exists. Build, lint, and `node --check server.cjs` pass, but unit/integration/E2E coverage must be introduced.

6. Agriculture/domain answers

Why is drainage emphasized over irrigation?

Dakshina Kannada receives heavy monsoon rain. Standing water and poor aeration increase root disease risk for areca, pepper, coconut, papaya, and other crops, while slopes also increase erosion and nutrient loss. Contour drains, channels, raised beds where suitable, ground cover, and avoiding low water-collecting pockets are therefore central. Irrigation still matters in dry periods, especially for sensitive growth stages, but it is not the district's dominant water-management problem.

Which crops and systems suit the area?

Paddy suits lower, banded fields with seasonal water management. Areca and coconut are signature plantation crops in suitable well-drained sites. Pepper, cocoa, banana, and in some cases vanilla can use vertical or understory space in plantations; rubber suits certain hillier taluks such as Belthangady and Sullya; fruits and vegetables fit site-specific drainage, shade, and market conditions. These are starting points, not prescriptions: soil testing, plot topography, local disease pressure, labour, water availability, and market plan should decide the final crop mix.

Explain the rice calendar and paddy practices.

The guide describes three local paddy seasons: Aati, Suggi, and Kolake. The interview point is that timing follows rainfall and field conditions rather than a universal calendar. Good practice includes timely nursery/transplanting or direct seeding as locally appropriate, clean bunds and drainage, weed/pest monitoring, balanced nutrient management based on soil tests, and harvest/drying contingency planning. Specific varieties, chemical inputs, and dosages should be confirmed with the KVK or Department of Agriculture because they vary by season, location, and current recommendation.

Pulses in paddy fallows can add a shorter-cycle crop, diversify income, improve ground cover, and—when legumes are properly managed—contribute to nutrient cycling. They should only be selected where post-paddy moisture, drainage, market access, and the next season's plan support them.

Why areca intercropping and diversification matter

Areca is locally prominent and can provide a structured canopy/support system, but it is a long-term, price-sensitive plantation crop. Pepper vines can climb support trees, cocoa can occupy shaded understory, and banana can provide early cash flow before the canopy closes. Vanilla may command high value but needs hand pollination and careful curing. Intercropping spreads weather, disease, and price risk and uses light/space more efficiently, but it can also create competition for nutrients/water, shade imbalance, labour peaks, and pest/disease pathways. It needs plot-specific spacing, drainage, nutrition, and marketing planning.

Pepper needs support and well-drained rooting conditions; waterlogging increases quick-wilt risk. Cocoa bean fermentation and controlled drying develop quality and prevent poor grade/mould. Rubber ties land up for years and tapping is constrained by heavy rain. Banana offers an earlier return; papaya needs excellent drainage because waterlogging can cause root problems. Humid vegetables are more exposed to fungal leaf diseases, so raised beds, airflow, sanitation, and early observation are sensible integrated measures.

Soil, pests, climate, post-harvest, and institutions

Compost, farmyard manure, green manure, and mulch add organic matter, improve soil structure and moisture/nutrient holding, protect bare soil, and reduce erosion pressure. Heavy monsoon rainfall can leach mobile nutrients and carry exposed topsoil downhill; soil cover, contour-aware management, organic matter, and soil-tested nutrient plans help reduce loss.

Integrated pest management means monitoring and using the least risky effective combination: sanitation, drainage, spacing, resistant material when available, physical/biological options, and carefully targeted label-compliant pesticide use only when justified. It is preferable to routine spraying because it can reduce unnecessary exposure, resistance pressure, input cost, and harm to beneficial organisms. It does not mean “never use a pesticide.” Any diagnosis, product selection, dosage, pre-harvest interval, or protective-equipment advice needs current expert and label verification.

Rain can spoil field-drying crops and cause mould. Monitor short-term forecasts, avoid unnecessary wet exposure, dry on clean protected surfaces, and keep covered drying or tarpaulins ready. KVK Dakshina Kannada can provide trials, demonstrations, training, and expert referrals. Farm investment must also reflect labour availability, remittance/urban land pressure, crop price variation, input cost, debt, and long payback periods. Diversification reduces dependence on any one crop or price cycle, while flexible planting/fertilizing based on actual monsoon onset, drainage maintenance, and soil moisture observation help climate resilience.

Schemes, market planning, technology, and sustainability

The English guide names PM-KISAN (eligible income support), PMFBY (crop insurance), Soil Health Card (soil testing/recommendations), PMKSY (water/irrigation efficiency), KCC (agricultural credit), and e-NAM (market discovery/trading platform). Eligibility, payment, crops, deadlines, premiums, and state implementation change, so farmers must confirm live details with official portals, RSK, or the Department of Agriculture before relying on them.

Market planning means estimating input/labour/transport costs before planting, identifying likely buyers, tracking prices, considering grade and storage realities, and timing sales without taking unmanageable storage or price risk. Technology can support weather checks, soil records, field mapping, pest-photo triage, market data, expert consultations, and irrigation scheduling; it should support—not replace—local observation and advice. Sustainable practices highlighted include integrated nutrient/pest management, trees and biodiversity, rainwater harvesting, pollinator protection, crop rotation, composting residues rather than burning, and diversification.

7. Responsible AI, data quality, and safety

Who owns and validates the content?

The prototype repository alone does not prove authorship or agronomic approval. The correct answer is: “The current guide needs a documented governance process.” In production, each section should have an author, local agronomist/KVK reviewer, approval date, source references, review interval, language reviewers, and a version history. High-risk content should expire or require review sooner.

Advice must distinguish broad education from a plot-specific recommendation. The assistant should state the conditions under which a guide statement may apply and request safe context where appropriate—crop, growth stage, symptom duration, rainfall/drainage condition—without collecting unnecessary PII. It must not diagnose disease or prescribe pesticide dosage from a photo/text alone. It should provide low-risk immediate steps, state uncertainty, cite a verified source, and direct the farmer to KVK/extension officers for diagnosis or label-regulated decisions.

How do you reduce hallucination and prompt injection?

Use a strict grounded answer mode, a minimum retrieval-confidence gate, an explicit no-answer/escalation response, and citations tied to returned source IDs. Test adversarial questions and source snippets that instruct the model to ignore policy. Retrieve only approved content server-side, delimiter-mark sources as untrusted data, and ensure the model can never claim a guide source that was not supplied. Evaluate whether source citations actually support each answer claim.

For multilingual quality, use a reviewed question set in each language, compare retrieval and answers for factual equivalence and naturalness, and test code-switched input. Do not instruct merely `Answer in tcy`; map language codes to explicit language names and validate output with human reviewers. If Tulu embeddings are poor, fall back transparently to lexical retrieval/curated navigation and prioritize a tested multilingual model; never silently pretend confidence is high.

The product now has feedback controls for “outdated,” “unsafe,” and “unclear,” with an optional comment and release/section traceability. Next, add “not in my language” and “needs expert help,” plus an escalation path to KVK/extension contacts. Protect consent, minimize farm data, and state that output is educational—not financial, legal, medical, or pesticide-label advice.

8. Testing, debugging, and delivery

How do you run it?

Run `npm install`, then `npm run dev` for Vite and `npm run api` in another terminal for the API. `npm run build` produces the static frontend bundle; `npm run preview` serves that build for a frontend preview; `npm run start` starts the Node server; `npm run lint` runs Oxlint. Create a local `.env` with `LLM_API_URL`, `LLM_API_KEY`, optional `LLM_MODEL`, `PORT`, and `FRONTEND_ORIGIN`; use `VITE_GUIDE_MANIFEST_URL` only for a deployed immutable-release manifest. Commit only the redacted `.env.example`, never a key.

Dev and API are separate because Vite's development server serves and hot-reloads frontend assets, while the Node process protects the provider secret and performs upstream calls. A production deployment can still expose them behind one domain/reverse proxy.

What tests would you add?

Unit-test `buildSections` for headings, preface handling, empty content, versioned IDs, and content fidelity. Unit-test hash freshness with changed heading/content/order. Wrap IndexedDB and the embedder behind interfaces, then use fake IndexedDB and a deterministic fake embedding vector to test cache reuse, version isolation, retrieval ranking, threshold behavior, and fallback after model failure. Test that a release does not replace the visible guide until all languages and indexes are ready, and that a manifest rollback restores the last healthy version.

For the API, inject/mock `fetch` and test valid response, missing question, invalid JSON, oversized body, invalid language/context, provider 401/500/timeout, malformed provider JSON, feedback validation/persistence, and safe error shapes. Add browser end-to-end tests for language switch/history policy, greeting without model load, question submission, disabled suggestions during submission, feedback submission, release swap/rollback, source display, keyboard focus, responsive layout, and semantic fallback. Run accessibility checks (keyboard, screen reader smoke test, contrast) and test low-end/slow-network first model download.

CI should run install with lockfile, lint, build, unit/integration/E2E tests, dependency/security scanning, and a deploy smoke test. Use content snapshots/parity assertions to catch missing translated sections. Version guides and model/index schema, deploy with a rollback artifact, and invalidate or namespace cached indexes when a guide/model changes.

9. Strong 60-second explanation

"Krishi is a multilingual farming guidance portal designed for Dakshina Kannada. It combines a readable local guide in English, Kannada, and Tulu with a question interface. For a question, the browser splits the selected Markdown guide by section, uses Transformers.js to create normalized embeddings, caches the section vectors in IndexedDB, and retrieves the most relevant local sections with cosine similarity. It then sends the question, language, release version, and local excerpts to a Node API, which keeps the LLM key server-side and generates a low-temperature answer. The guide content is released through an immutable manifest: the app downloads and indexes a complete new release before atomically switching, so updates and rollbacks do not interrupt users. Feedback is tied to the exact release and retrieved sections. The remaining priorities are a real distinction between Guide and Latest modes, verifiable citations, visible error/retry UX, full language parity, rate limiting/timeouts/tests, and agronomist-reviewed safety governance."